LLM FROM SCRATCH · STUDY GUIDE

Chhotu GPT, andar se.

Ek language model jo Hindi, Marathi aur Hinglish likhta hai. Tokenizer se quantization tak har hissa haath se bana. Ye guide batati hai wo kaise bana, har concept kya hai, code mein kahan hai, aur interview mein kaise samjhana hai.

12.2 M parameters, 6 transformer blocks	**8.37 → 1.88** loss, 4000 iterations mein	**10.9 M** training tokens, 3 bhashayein
4,096 tokens ki vocabulary, khud ka BPE	**12.2 MB** int8 mein size, loss mein fark 0	**169** concepts, 153 code ki line ke saath

Saare numbers isi project ke run se naape gaye hain · Apple M1 Pro

Is guide ko kaise padhna hai

Pehle 2 page poori kahani dete hain. Phir 9 chapter, har ek ek file ke baare mein. Aakhir mein saare 169 concepts ki list aur interview ke sawaal-jawab.

0 Poora project ek nazar mein
1 Autograd: network seekhta kaise hai
2 Tokenizer: text se numbers
3 Data: kya padhaya
4 Model: transformer ke andar
5 Training: random se bhasha tak
6 Generation: numbers se text
7 LoRA: sasta fine-tune
8 Quantization: phone ka size
9 X-ray: andar jhaankna
10 Famous repos ke saamne hum
11 Saare 169 concepts
12 Interview: sawaal aur jawab
13 Ek page ki cheat sheet

SEEDHI BHASHA MEIN

Sabse simple shabdon mein. Jaldi mein ho to sirf ye padho.

INTERVIEW MEIN BOLO

Wo line jo interview mein seedhe English mein boli ja sakti hai.

CODE MEIN KAHAN

File aur wo naam jahan ye cheez bani hai. Kholo, search karo.

SACH BAAT

Jahan model kamzor hai ya result pakka nahi. Ye bolna tumhe mazboot dikhata hai.

Ye project kya hai, 4 line mein

- Ek **decoder-only transformer** (GPT jaisa), PyTorch mein, har layer khud likhi.
- Kaam ek hi: **agla token predict karna**. Baaki sab isi se nikalta hai.
- Hindi aur Marathi ki chhoti kahaniyon aur Hinglish chat pe train hua.
- Ye Gemma ya ChatGPT ki jagah nahi lega. Ye **seekhne ka model** hai: wahi architecture, chhota size.

9 files, 9 lessons

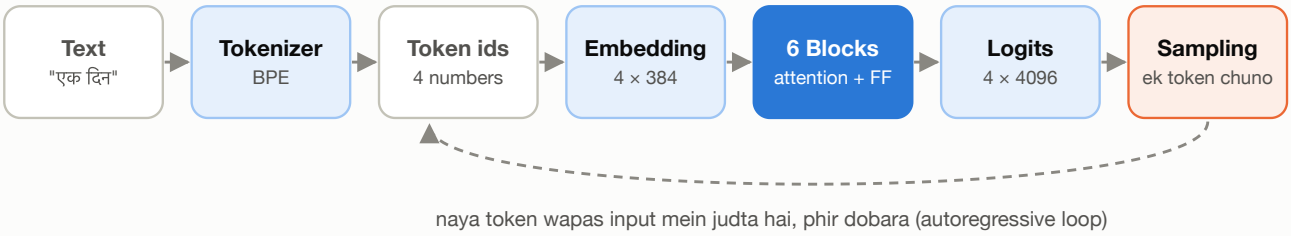
#	FILE	EK LINE MEIN	SABSE ZAROORI CONCEPT
1	`autograd.py`	Network seekhta kaise hai	Backpropagation
2	`tokenizer.py`	Text ko numbers banana	Byte-pair encoding
3	`data.py`	Training ka text tayyar karna	Train / val split
4	`model.py`	Transformer	Self-attention
5	`train.py`	Training loop	Cross-entropy loss, AdamW
6	`generate.py`	Text banana	Sampling, KV cache
7	`lora.py`	Fine-tuning	Low-rank adaptation
8	`shrink.py`	Model chhota karna	Quantization
9	`xray.py`	Andar dekhna	Attention maps

0 Poora project ek nazar mein

ek token ka safar, shuru se aakhir tak

LLM jaadu nahi hai. Wo ek function hai: **ab tak ke tokens andar, agle token ki probability bahar.** Text banta hai kyunki hum ye function baar-baar chalate hain.

Text se agle token tak

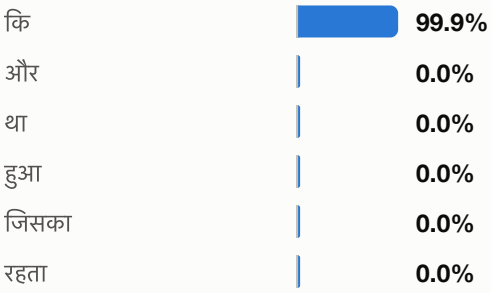


Padhne ka tareeka: baayein se daayein. Neeche wala teer loop hai: naya token input mein judta hai aur sab dobara chalta hai.

Model sach mein kya "sochta" hai

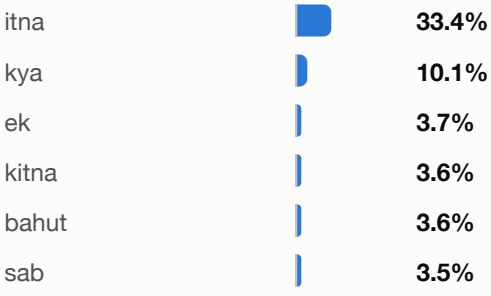
Ye do chart hamare train hue model ke asli output hain. Har bar ek possible agla token hai, lambai uski probability.

"एक दिन राजू ने अपनी माँ से कहा" ke baad



Model lagbhag pakka hai: Hindi mein "कहा" ke baad "कि" hi aata hai. Usne grammar seekh li.

"aaj office mein" ke baad



Yahan kai jawab sahi ho sakte hain, to bharosa bata hua hai. Ise **entropy** zyada hona kehte hain.

SEEDHI BHASHA MEIN

Model ek shabd nahi "jaanta". Wo har baar 4,096 tokens ko number deta hai, aur hum unme se ek chunte hain. Jab wo pakka hota hai to ek bar lamba hota hai; jab nahi hota to kai chhote.

1 Autograd: network seekhta kaise hai

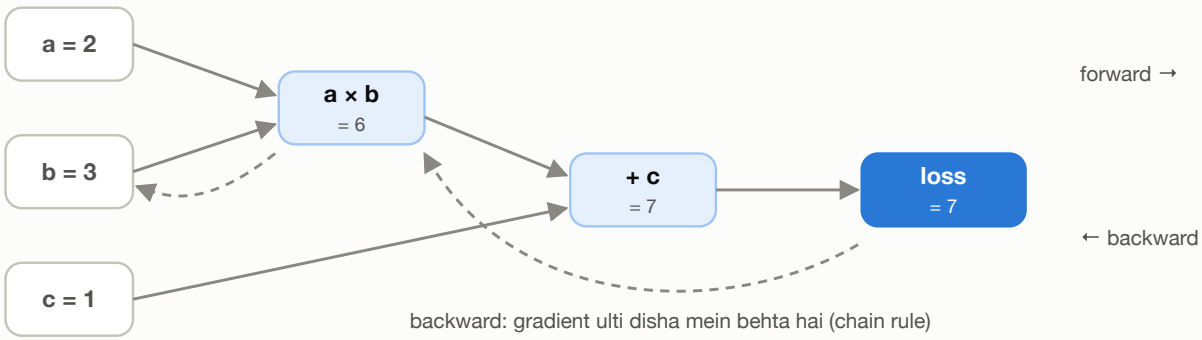
`autograd.py` · pure Python, koi library nahi

Poori deep learning ek hi loop hai: andaza lagao, dekho kitna galat tha, har weight ko thoda sahi disha mein khiskao. Ye chapter wo loop sabse chhote roop mein banata hai.

SEEDHI BHASHA MEIN

Socho tum andhere mein pahaad se neeche utar rahe ho. Tum pair se zameen ka dhalaan mehsoos karte ho aur neeche ki taraf ek kadam lete ho. **Dhalaan = gradient. Kadam ka size = learning rate. Oonchai = loss.** Bas yahi training hai.

Computation graph: number kaise bane, uska record



Forward: a aur b se loss tak hisaab. **Backward:** loss se peeche, har number se poochho "tumhe badalne se loss kitna badlega". Yahi backpropagation hai, aur ye sirf chain rule hai.

Chaar step jo kabhi nahi badalte

STEP	KYA HOTA HAI	CODE MEIN
Forward pass	Input ko network se guzaar kar prediction nikalna	`net(x)`
Loss	Ek number: prediction kitni galat hai	`(net(x) - y) ** 2`
Backward pass	Har weight ka gradient nikalna	`loss.backward()`
Update	Weight ko gradient ke ulte taraf khiskana	`p.data -= lr * p.grad`

NAAPA HUA

Gradient check: hamara backprop 0.084245 deta hai, aur "nudge karke dekho" tareeka bhi 0.084245. Dono barabar, yaani engine sahi hai.

XOR: 1000 step mein loss 0.478 se 0.0000. Predictions [0, 1, 1, 0].

SACH BAAT

Pehli baar learning rate 0.1 pe loss ghatne ki jagah **infinity** ho gaya (divergence). 0.05 pe theek chala. Ye sabse aam training bug hai: step itna bada ki ghaati ke paar kood gaye.

INTERVIEW MEIN BOLO

"Backpropagation is just the chain rule applied over a computation graph in reverse topological order. I implemented a scalar autograd engine myself and verified it with a finite-difference gradient check before moving to PyTorch."

CODE MEIN KAHAN

`class Value` ek number + uska gradient. `_backward` har operation ka local chain rule. `def backward` poore graph pe ulta chalna. `class MLP` wahi cheez jo har transformer block ke andar hai.

2 Tokenizer: text se numbers

`tokenizer.py` · byte-pair encoding (BPE), scratch se

Model akshar nahi padh sakta, sirf numbers. Tokenizer tay karta hai ki "ek tukda" kya hai. Ye faisla model ki quality pe seedha asar daalta hai.

TAREEKA	FAYDA	NUKSAAN
Har akshar ek token	Vocabulary bahut chhoti	Sequence bahut lambi, seekhna mushkil
Har shabd ek token	Sequence chhoti	Lakhon shabd, naya shabd aaya to fail
Subword (BPE)	Beech ka raasta, har LLM yahi leta hai	Tokenizer ko pehle train karna padta hai

BPE teen step mein

Shuruaat न म स ◌़ त ◌े har tukda alag (asal mein bytes; ek Devanagari akshar = 3 bytes)

Merge 1 न म स् त ◌े sabse zyada saath aane wala pair "स" + "◌़" ek ban gaya

Merge 2 न म स् ते phir agla sabse aam pair

… 3,835 merges नमस्ते aam shabd poora ek token ban jaata hai

Ye tasveer samjhane ke liye saral ki gayi hai; asal mein shuruaat bytes se hoti hai. 256 bytes + 3,835 merges + 5 special tokens = **4,096** ki vocabulary.

Hamara tokenizer asli vaakyon ko aise kaat-ta hai

Hindi 76 bytes → 9 tokens

एक | ▁दिन | ▁राज | ू | ▁ने | ▁अपनी | ▁माँ | ▁से | ▁कहा

Marathi 115 bytes → 8 tokens

आर्यन | ला | ▁समुद्र | किन | ाऱ्यावर | ▁खेळायला | ▁आवडत | ▁असे

Hinglish 40 bytes → 11 tokens

kya | ▁chal | ▁raha | ▁hai | ▁b | hai | , | ▁weekend | ▁ka | ▁plan | ?

▁ ka matlab space. Aam shabd ek token ("अपनी", "weekend"). Kam aane wale shabd tukdon mein: "राजू" = "राज" + "ू", aur "bhai" = "b" + "hai". BPE ko shabd ka matlab nahi pata, sirf ginti pata hai.

Ek token mein kitne akshar samaate hain (300 documents pe naapa)



Zyada = behtar compression. Hinglish Latin script mein hai (1 byte per akshar), Devanagari 3 bytes leta hai par tokenizer use wapas jod deta hai.

BYTE-LEVEL KYUN

256 bytes se shuru karne ka matlab: duniya ka koi bhi text encode ho sakta hai. Emoji, naya shabd, galat spelling, kuch bhi. "Unknown token" kabhi nahi aata.

SPECIAL TOKENS

`<|bos|>` `<|eos|>` document ki shuruaat aur ant. `<|hi|>` `<|mr|>` `<|hing|>` bhasha ka tag: isse hum model ko batate hain kis bhasha mein likhna hai.

INTERVIEW MEIN BOLO

"I wrote a byte-level BPE tokenizer. It starts from 256 bytes so nothing is ever out of vocabulary, then greedily merges the most frequent adjacent pair. I pre-tokenize into words and keep a pair-to-words index, so each merge only touches the words that contain that pair."

3 Data: model ko kya padhaya

`data.py` · teen open datasets, 92 MB download

Model utna hi achha hota hai jitna uska data. Chhota model poora internet nahi seekh sakta, par chhoti vocabulary wale text se grammar aur seedha logic seekh sakta hai. Yahi **TinyStories** ka idea hai.

Documents ki ginti

Hindi kahaniyan	**17,893**
Marathi kahaniyan	**11,184**
Hinglish chat	**71,306**

Hinglish ke documents bahut hain kyunki har ek sirf ek sawaal-jawab hai.

Kul akshar (million)

Hindi kahaniyan	**18.4 M**
Marathi kahaniyan	**12.0 M**
Hinglish chat	**11.6 M**

Akshar ke hisaab se teeno bhashayein lagbhag barabar hain.

Har document ka dhaancha

<|bos|> <|hi|> एक बार की बात है, एक छोटे से गाँव में… <|eos|>

Har document bhasha ke tag se shuru hota hai. Isliye generation ke waqt sirf tag badal kar bhasha chuni ja sakti hai. Ise **conditional generation** kehte hain.

CHEEZ	NUMBER	MATLAB
Train tokens	10.87 M	Jis pe model seekhta hai
Val tokens	0.22 M	2% documents alag rakhe. Model inhe kabhi nahi dekhta.
Epochs	4.5	Training mein poora data itni baar dekha gaya
File format	uint16	Har token 2 bytes; vocabulary 65,536 se chhoti hai

VAL SPLIT KYUN

Exam mein wahi sawaal aaye jo ratte the to pata nahi chalta ki samjha ya nahi. Val data wo naye sawaal hain. Agar train pe achha aur val pe kharab, to model ne ratta maara: ise **overfitting** kehte hain.

SACH BAAT

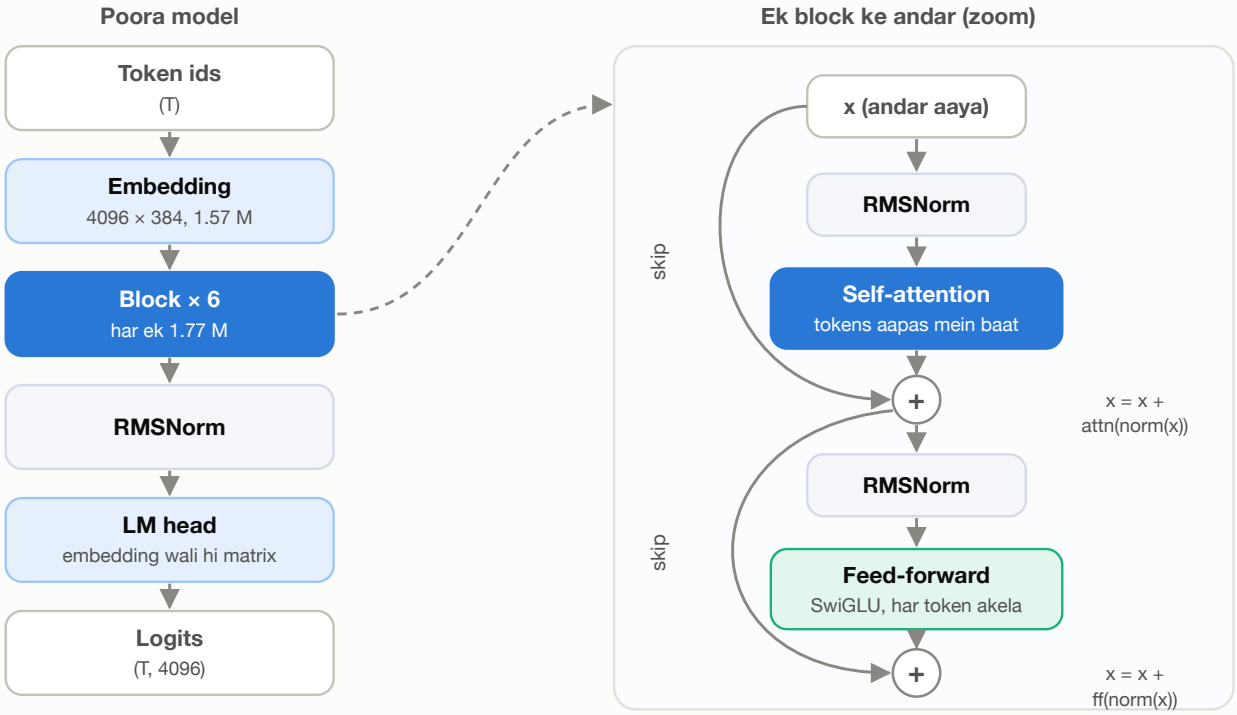
Har dataset ka sirf pehla hissa liya hai (Hindi file 5.4 GB ki hai, hamne 48 MB liya). Aur ye kahaniyan GPT-4o-mini ki banayi hui hain, insaanon ki nahi. Isliye model "bachchon ki kahani" ki bhasha seekhta hai, aam Hindi nahi.

4 Model: transformer ke andar

`model.py` · 12,194,688 parameters

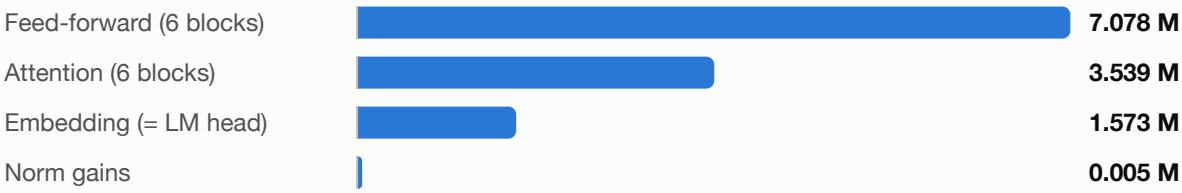
Transformer do cheezon ko baar-baar dohraata hai: **attention** (tokens ek doosre se jaankari lete hain) aur **feed-forward** (har token akele sochta hai). Dono ke saath ek skip connection.

Architecture



Skip connection: block x ko badalta nahi, usme kuch *jodta* hai. Isi "+" ke raaste gradient seedha neeche tak pahunchta hai, isliye 6 ya 60 layers train ho paati hain.

Parameters kahan hain (asli model se gine)

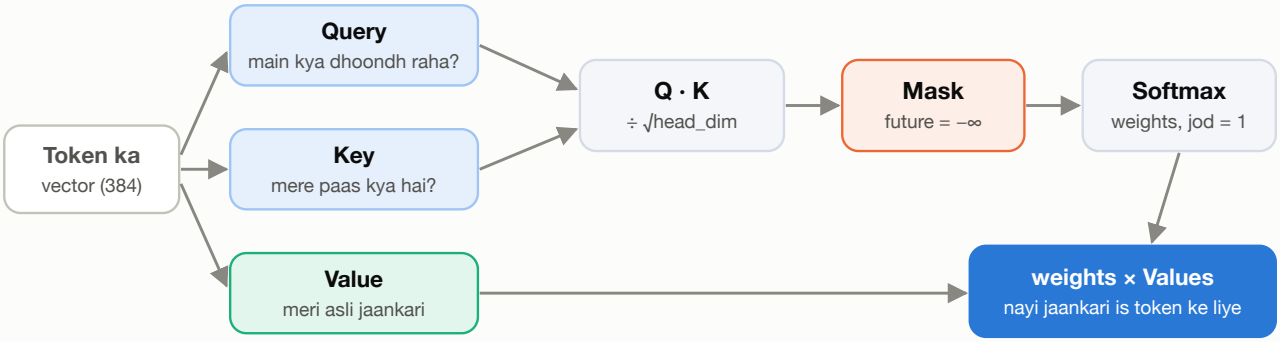


Kul 12,194,688. Sabse zyada feed-forward mein, jahan model ki "yaaddasht" rehti hai. Embedding aur LM head ek hi matrix hain (**weight tying**), isliye ek baar gine.

SEEDHI BHASHA MEIN

Socho ek meeting hai. **Attention** = sab ek doosre ki baat sunte hain aur zaroori cheez note karte hain. **Feed-forward** = har koi apne notes pe akele sochta hai. Ye do kaam 6 baar hote hain, aur har baar samajh thodi gehri hoti hai.

Attention, tukdon mein



Ye sab 6 heads mein ek saath hota hai, phir sab jud kar wapas ek vector banta hai.

Query sawaal hai, **Key** label hai, **Value** asli maal hai. Jis token ki key meri query se milti hai, uski value mujhe zyada milti hai.

√HEAD_DIM SE BHAAG KYUN

Bade vectors ka dot product bada hota hai. Bade numbers pe softmax ek hi token ko 100% de deta hai aur baaki sab 0. Tab gradient mar jaata hai. Bhaag dene se numbers kaabu mein rehte hain.

MULTI-HEAD KYUN

Ek head ek hi rishta pakad sakta hai. 6 heads alag-alag cheezein dekhte hain: kaun subject hai, kaunsa quote khula hai, pichhla shabd kya tha.

Causal mask: future dekhna mana hai

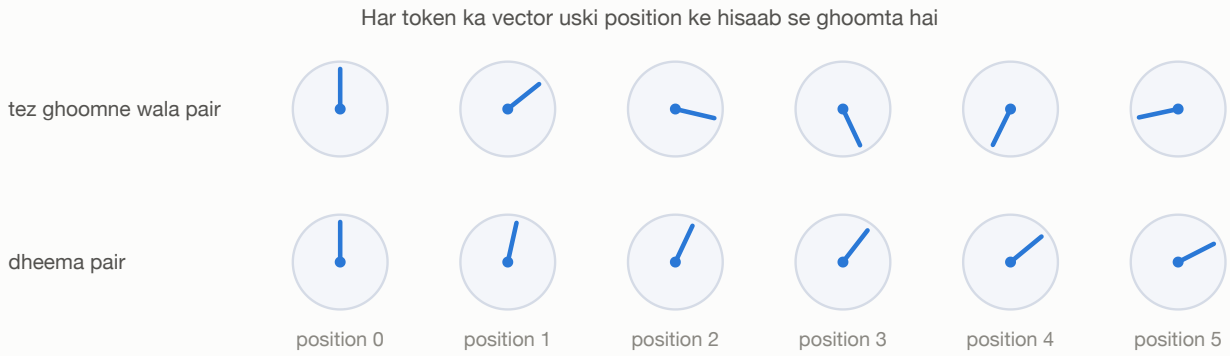
	एक	दिन	राजू	ने	कहा
एक	✓	×	×	×	×
दिन	✓	✓	×	×	×
राजू	✓	✓	✓	×	×
ने	✓	✓	✓	✓	×
कहा	✓	✓	✓	✓	✓

Row = jo token soch raha hai
Column = jise wo dekh sakta hai

"राजू" sirf "एक दिन राजू" dekhta hai.
"ने" aur "कहा" uske liye future hain.

Isi wajah se ek hi pass mein har position pe training ho jaati hai, bina cheating ke.

RoPE: model ko order kaise pata chalta hai



Attention ko apne aap order nahi pata: shabd ulat-palat karo to wahi jawab dega. RoPE har vector ko uski position ke angle se ghumata hai, ghadi ki suiyon ki tarah alag-alag speed pe. Do tokens ka score phir sirf unke beech ki **doori** pe depend karta hai.

Design ke faisle: purana GPT-2 banam aaj ke LLM

CHEEZ	GPT-2 (2019)	HAMNE KYA LIYA	KYUN
Position	Seekhi hui table	**RoPE**	Relative position, alag table nahi chahiye
Normalization	LayerNorm	**RMSNorm**	Sasta, utna hi achha
Norm kahan	pre-norm	**pre-norm**	Skip ka raasta saaf rehta hai
Feed-forward	GELU, 4x	**SwiGLU**	Llama aur Gemma yahi use karte hain
Bias	haan	**nahi**	Fark nahi padta, parameters bachte hain
Embedding / head	tied	**tied**	1.57 M parameters bachte hain

INTERVIEW MEIN BOLO

"It's a pre-norm decoder-only transformer with the modern Llama-style choices: RoPE instead of learned positions, RMSNorm instead of LayerNorm, SwiGLU instead of GELU, no biases, and tied input and output embeddings. I wrote attention by hand instead of calling scaled_dot_product_attention so I could record and inspect the attention weights."

CODE MEIN KAHAN

`class CausalSelfAttention` poora attention. `def apply_rope` rotation. `class RMSNorm`, `class FeedForward`, `class Block`. `def cross_entropy` loss, haath se. `class Config` saare hyperparameters ek jagah.

Hyperparameters

384	6	6	256
d_model: har token ke vector ki lambai	n_layer: blocks	n_head: har block mein heads (har ek 64 ka)	ctx: ek baar mein kitne tokens

KAISE PATA CODE SAHI HAI

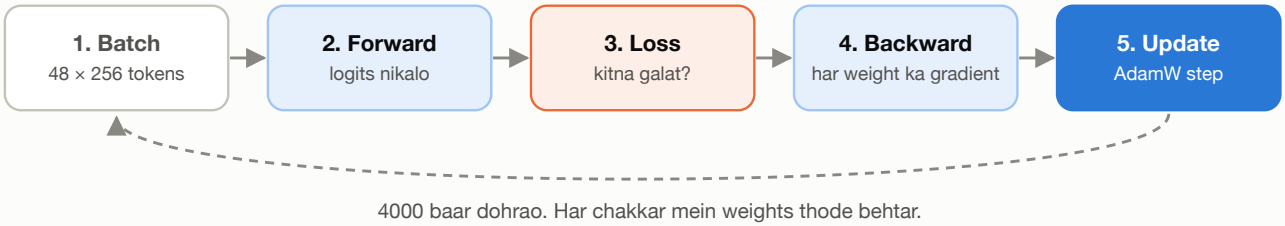
`check.py` ye sab test karta hai, aur sab pass hain:

- **Causality:** baad ka token badalne se pehle ki predictions nahi badalti. Mask sahi hai.
- **Loss:** haath se likha cross-entropy PyTorch wale ke barabar hai.
- **KV cache:** ek-ek token dene pe wahi logits aate hain jo poori sequence dene pe.
- **LoRA:** naya adapter kuch nahi badalta; merge karne ke baad output wahi rehta hai.

5 Training: random se bhasha tak

`train.py` · 4000 iterations, ~50 minute, Mac ke GPU pe

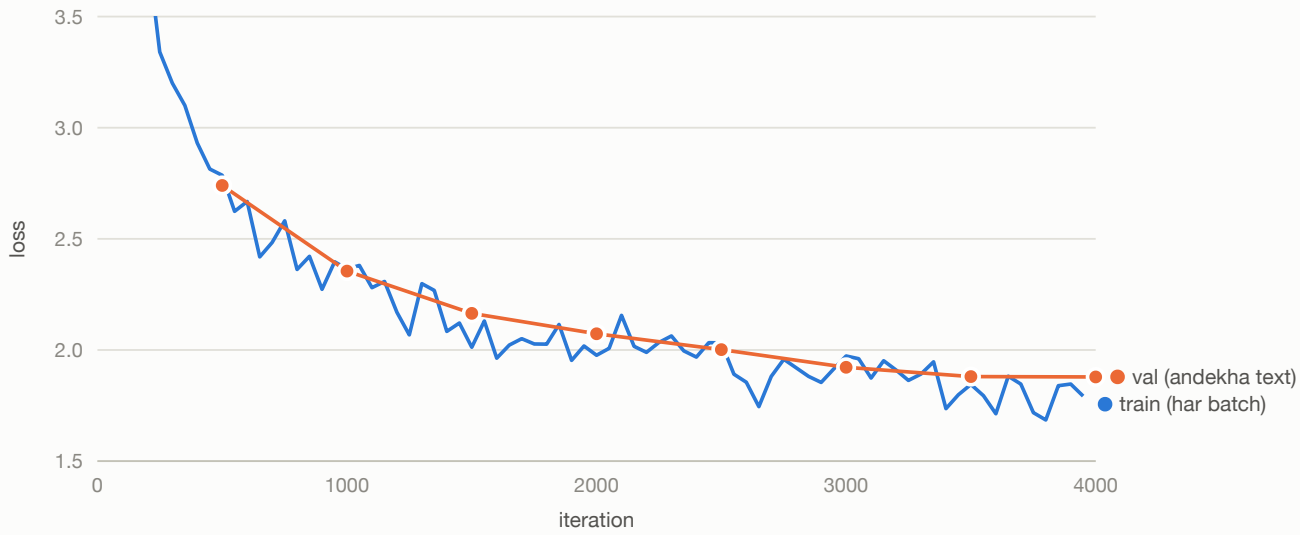
Training **self-supervised** hai: kisi ne kuch label nahi kiya. Har position ka sahi jawab text ka agla token hi hai.



Training ke 6 hisse, aur har ek kyun hai

HISSA	KYA KARTA HAI	NA HO TO
Cross-entropy loss	Sahi token ko di gayi probability ka –log	Seekhne ka koi signal nahi
AdamW	Har weight ka apna step size + momentum	Sada SGD se transformer bahut dheere seekhta hai
Weight decay	Weights ko zero ki taraf kheenchta hai	Ratta maarne ka khatra badhta hai
Warmup + cosine	Learning rate ka time-table	Shuru mein training bigad sakti hai
Gradient clipping	Bahut bade gradient ko 1.0 pe kaat deta hai	Ek kharab batch model bigaad sakta hai
Dropout	10% numbers random zero	Model kuch hi raaston pe tik jaata hai

Loss curve: model seekh raha hai



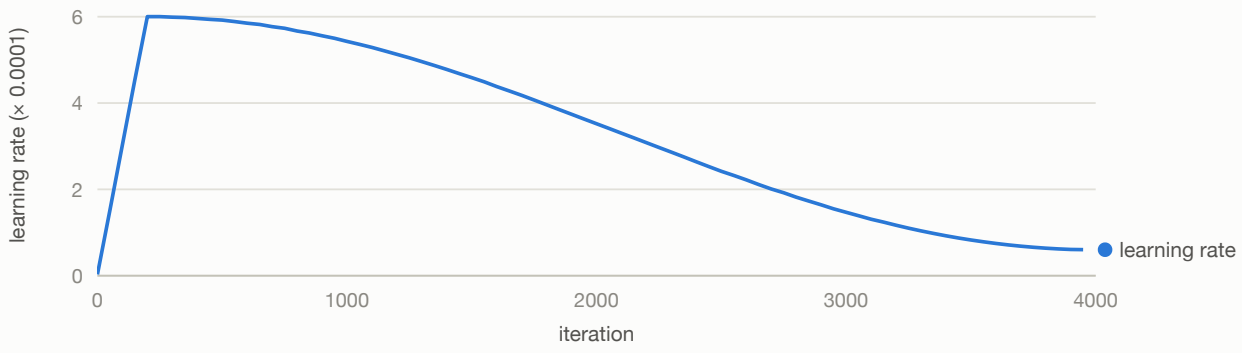
Shuruaat 8.37 se hui (chart ke upar, bahar). Neela = har batch ka loss, hilta hai kyunki har batch alag hai. Naarangi = andekhe text pe loss, har 500 iteration pe. Dono saath gir rahe hain aur aakhir mein sirf 0.19 ka fark hai: **overfitting nahi hui**.

8.37	1.69	1.88	6.5
pehla loss. ln(4096) = 8.32 hona chahiye tha. Sahi.	aakhri train loss	aakhri val loss	perplexity: 4,096 mein se ~6 ke beech confuse

PEHLA LOSS 8.32 KYUN HONA CHAHIYE

Bina seekha model har token ko barabar chance deta hai: 1/4096. Uska loss −ln(1/4096) = 8.32. Agar pehla loss isse bahut alag aaye to code mein bug hai. Ye sabse sasta sanity check hai, aur interview mein ye batana bahut achha lagta hai.

Learning rate: pehle warmup, phir cosine decay



Pehle 200 step mein dheere badhta hai (random network ke shuruaati gradients jangli hote hain), phir cosine curve pe ghat-ta hai taaki model aakhir mein sthir ho.

TEACHER FORCING

Training mein model ko hamesha *sahi* pichhle tokens milte hain, uske apne galat andaze nahi. Aur causal mask ki wajah se 256 tokens ki ek sequence 256 examples ban jaati hai.

PERPLEXITY

exp(loss). 6.5 ka matlab: har kadam pe model utna unsure hai jitna 6-7 barabar options mein se chunte waqt hota. Random model ka 4,096 hota.

INTERVIEW MEIN BOLO

"Training is next-token prediction with cross-entropy. I use AdamW with weight decay only on the matrices, linear warmup then cosine decay, and gradient clipping at 1.0. I sanity-checked that the initial loss was ln of the vocab size, and I track validation loss on held-out documents to catch overfitting. It went from 8.37 to 1.88."

SACH BAAT

Val loss aakhir tak gir raha tha, yaani training rokne ka waqt nahi aaya tha: hamne bas 4000 pe rok diya. Zyada iterations ya zyada data se model aur behtar hoga.

6 Generation: numbers se text

`generate.py` · sampling aur KV cache

Model sirf probabilities deta hai. Unme se token *kaise* chunte hain, isse output ka mizaaj tay hota hai.

Temperature ka asar: wahi model, wahi prompt ("aaj office mein")

temperature 0.5
safe, ek hi jawab

itna	86%
kya	8%
ek	1%
kitna	1%
bahut	1%
sab	1%

temperature 1.0
model ka asli bharosa

itna	33%
kya	10%
ek	4%
kitna	4%
bahut	4%
sab	3%

temperature 1.5
creative, risky

itna	10%
kya	4%
ek	2%
kitna	2%
bahut	2%
sab	2%

Logits ko temperature se bhaag dete hain. Kam temperature pe sabse likely token aur bada ho jaata hai; zyada pe sab barabar hone lagte hain.

TAREEKA	KYA KARTA HAI	KAB
Greedy	Hamesha sabse likely token	Jab ek hi sahi jawab chahiye. Aksar repeat karta hai.
Temperature	Distribution ko sharp ya flat karta hai	0.7 – 0.9 aam taur pe
Top-k	Sirf k sabse likely tokens rakhta hai	Bekaar tokens kaatne ke liye

TAREEKA	KYA KARTA HAI	KAB
Top-p (nucleus)	Utne tokens jinka jod p ho	Jab model pakka ho to kam, na ho to zyada options
Repetition penalty	Aa chuke tokens ko dabata hai	Loop rokne ke liye

KV cache: wahi jawab, kam kaam

SEEDHI BHASHA MEIN

Bina cache: har naye token ke liye model poori kahani shuru se dobara padhta hai. Cache ke saath: purane tokens ke keys aur values yaad rakhe jaate hain, aur sirf naya token process hota hai. Jawab bilkul same, `check.py` ye test karta hai.

Speed, is machine pe (150 tokens, CPU)



3.2x tez. Sequence jitni lambi, fayda utna zyada.

Model ne asal mein kya likha

> **एक दिन राजू ने अपनी माँ से कहा** कि वह जंगल में घूमने जाएगा। सभी जानवर बहुत उत्साहित थे। राजू ने सोचा, "क्यों न मैं भी उनके साथ खेलूं?"

> **एकदा एक लहान मुलगा** होता, ज्याचे नाव आर्यन होते. आर्यनला समुद्रकिनाऱ्यावर खेळायला खूप आवडत असे. एक दिवस, तो समुद्राच्या किनाऱ्यावर गेला.

> **kya chal raha hai bhai?** → kuchh nahin yaar, bas aaram karna hai.

> **weekend ka kya plan hai?** → bas ghar pe relax karna hai, aaram karne ke liye.

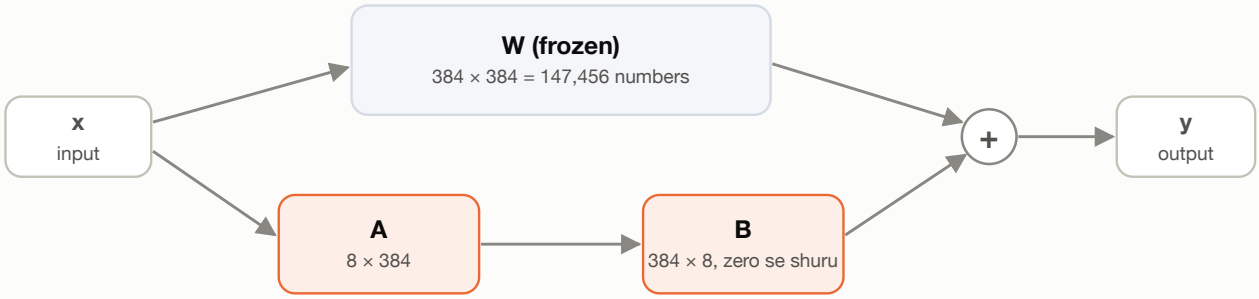
SACH BAAT

Grammar theek hai, par kahani ka logic toot-ta hai: Raju akela jaane ki baat karta hai, phir achanak "sabhi jaanwar utsaahit" ho jaate hain. Model shabdon ka pattern jaanta hai, duniya nahi. Isi ko bade models mein **hallucination** kehte hain.

7 LoRA: 1% se kam weights se fine-tune

`lora.py` · 20 sawaal-jawab, 600 steps

Pre-training bhasha sikhati hai. **Fine-tuning** us model ko ek kaam, domain ya personality ke liye dhaalti hai. LoRA ye bina original weights chhue karta hai.



Sirf A aur B train hote hain: 6,144 numbers, W ka 4%. Baad mein W + B·A jodkar ek matrix.

B zero se kyun shuru: taaki pehle step pe B·A = 0 ho aur model bilkul pehle jaisa chale. Training dheere-dheere badlaav jodti hai.

	FINE-TUNE SE PEHLE	BAAD MEIN
Train hue parameters	—	**110,592** of 12.2 M (0.91%)
Persona loss	4.58	**0.32**
General val loss	1.829	1.888 (+0.06, thoda bhoola)
"tera naam kya hai?"	mera toh bas normal hi hai.	**mera naam Chhotu GPT hai.**
"tu kaun hai?"	—	main ek chhota GPT hun, scratch se banaya gaya.
File ka size	48.8 MB (poora model)	**0.45 MB** (sirf adapter)

SEEDHI BHASHA MEIN

Poori kitaab dobara likhne ki jagah, uske haashiye mein chhote notes chipka do. Kitaab wahi rehti hai, notes use naya kaam sikha dete hain. Notes ki file 100 guna chhoti hai.

RANK KYA HAI

A aur B kitne "patle" hain. Rank 8 ka matlab badlaav 8 dishaon tak seemit hai. Maanyata ye hai ki ek naye kaam ke liye zaroori badlaav simple hota hai, chahe model khud complex ho.

SACH BAAT: TEEN CHEEZEIN JO GALAT GAYI

- **Bhoolna.** Pehli koshish mein general val loss 2.70 se 3.19 ho gaya (catastrophic forgetting). Har step mein purane data ka ek batch jodne se (**replay**) ye kaafi kam hua.
- **Kam training.** 300 steps pe model ne apna naam "Mhhotu GTP" likha. 600 pe sahi aaya.
- **Ratta.** Persona aam baat mein ris gaya: "kya chal raha hai bhai?" ka jawab aaya "kuchh nahi yaar, bas phone pe scratch se banaya gaya." 20 examples samajhne ke liye kam hain.

INTERVIEW MEIN BOLO

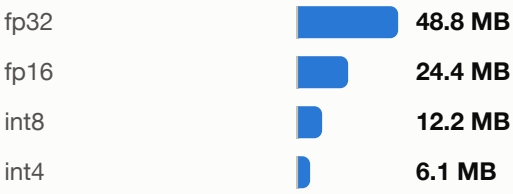
"LoRA freezes the pretrained weights and learns a low-rank update B times A on the attention matrices. B is initialized to zero so training starts from the original model. I trained 0.91% of the parameters, the adapter is 0.45 megabytes, and after training I merge it back so inference cost is unchanged. I also saw catastrophic forgetting and reduced it by replaying original data."

8 Quantization: phone ke size tak

`shrink.py` · weights ko kam bits mein rakhna

Weights 32-bit floats mein train hote hain. Unhe itni baareeki ki zaroorat nahi hoti. Kam bits mein rakhne se model chhota hota hai; sawaal ye hai ki kitna nuksaan hota hai.

Size

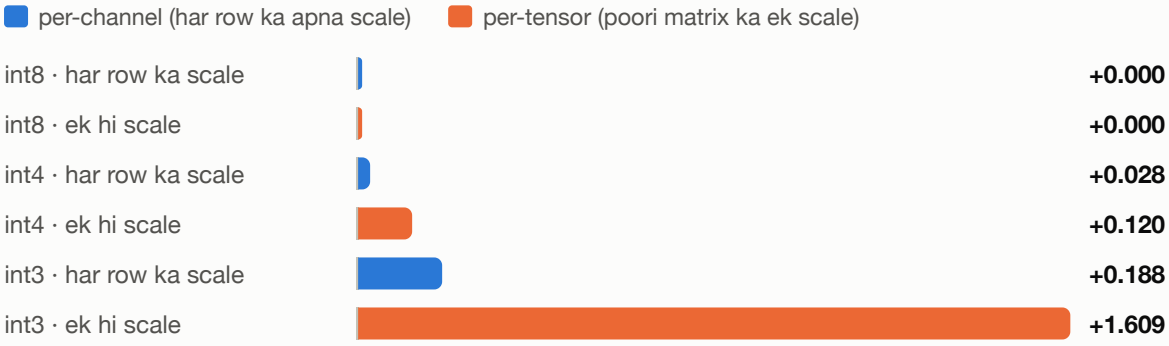


Parameters × bits ÷ 8. Ye weights ka hisaab hai, file format ka nahi.

SEEDHI BHASHA MEIN

3.14159 ko 3.1 likhna. Har number thoda galat ho jaata hai par jagah bahut bachti hai. int8 mein har weight −127 se 127 ke beech ka ek poora number ban jaata hai, saath mein ek **scale** jo use wapas float banata hai.

Nuksaan: val loss kitna badha (fp32 ke mukable, same batches)



int8 tak muft hai. 4-bit pe fark dikhta hai, aur har row ka apna scale rakhna (per-channel) 4 guna behtar hai. 2-bit pe model poori tarah toot gaya (loss 8.6, random jaisa), isliye wo chart mein nahi hai.

TERM	MATLAB
PTQ (post-training)	Train hone ke baad quantize karna. Hamne yahi kiya.
QAT (quantization-aware)	Training ke dauran hi quantization ka asar dikhana. Zyada mehnat, kam nuksaan. Gemma ka QAT version yahi hai.
Per-tensor	Poori matrix ka ek scale. Simple.
Per-channel	Har row ka apna scale. Chhote weights wali row bade weights wali row ki wajah se nahi pisti.

SACH BAAT

Pehli baar chaaron rows mein loss bilkul same aaya (1.8448) aur laga code mein bug hai. Jaanch ki to pata chala int8 ki galti weight ke size ka sirf ~1% hai, isliye loss sach mein nahi hilta. Tab 4, 3 aur 2 bit jode taaki dikhe ki kahan toot-ta hai. **Seekh: "kuch nahi badla" ko pehle shak se dekho, phir naap ke maano.**

INTERVIEW MEIN BOLO

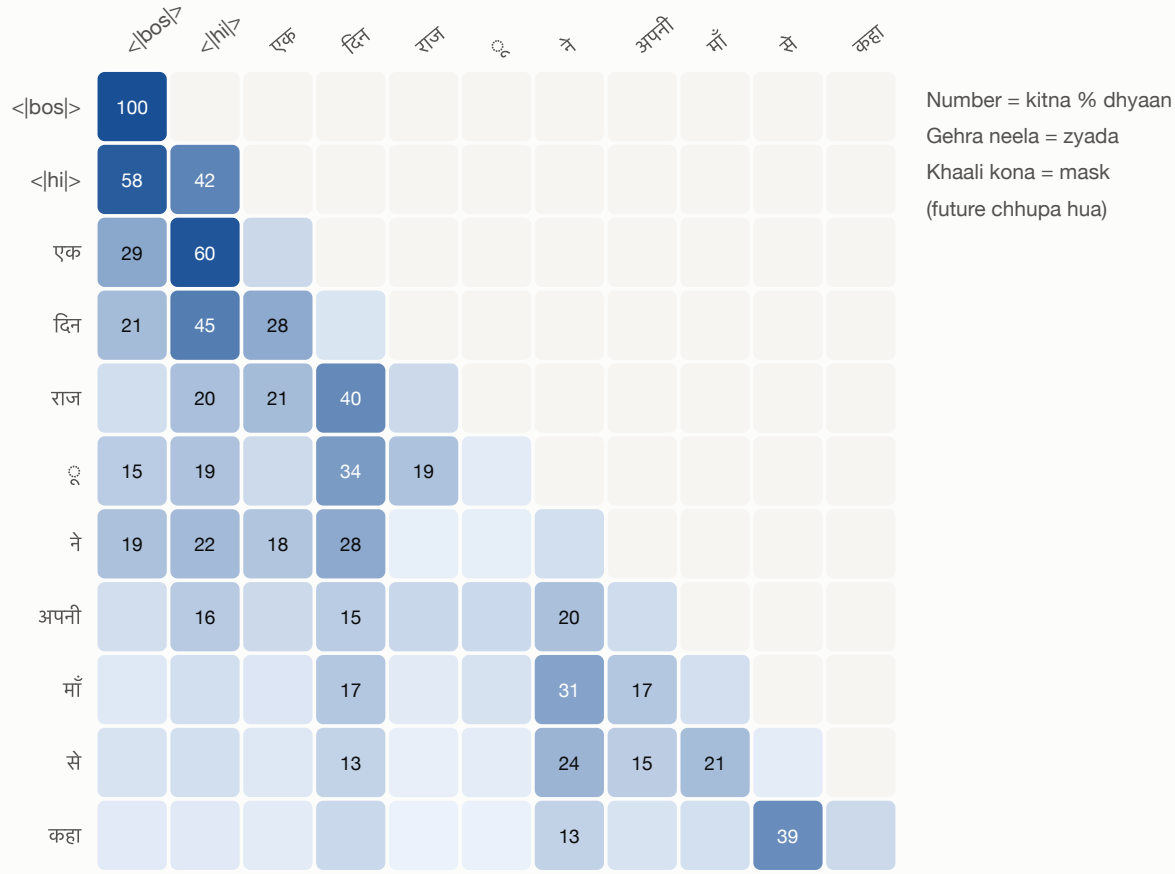
"I did post-training quantization and measured validation loss on identical batches. int8 was free, 12 megabytes instead of 49. At 4 bits per-channel scaling lost 0.03 while per-tensor lost 0.12, and at 2 bits the model collapsed. So the practical floor for this model is around 4 bits with per-channel scales."

9 X-ray: model ke andar jhaankna

`xray.py` · tokens, belief, attention

Jo model tum dekh sakte ho, wahi tum samjha sakte ho. Ye asli attention map hai, hamare train hue model se, jab wo "एक दिन राजू ने अपनी माँ से कहा" padh raha tha.

Layer 3 ka attention (6 heads ka average)



Padhne ka tareeka: har row ek token hai, aur us row ke numbers batate hain wo kis pichhle token ko kitna dekh raha hai. Aakhri token "कहा" sabse zyada in pe dhyaan deta hai: **से 39%, ने 13%, दिन 10%**. Upar-daayein ka khaali kona causal mask hai.

SEEDHI BHASHA MEIN

"कहा" ke baad kya aayega ye tay karne ke liye model "से" aur "ने" ko dekhta hai: yaani wo vaakya ka dhaancha pakad raha hai (kisne, kisse). Kisi ne use grammar nahi sikhayi; ye agla token predict karte-karte khud ubhra.

SACH BAAT

Attention map dekhkar kahani banana aasaan hai aur galat bhi ho sakta hai. Ye 6 heads ka average hai, aur attention weight "kaaran" nahi hai, sirf "kahan dekha". Interview mein ise **interpretability ka pehla kadam** kaho, saboot nahi.

CHALAO

`python xray.py --lang hi --prompt "एक दिन राजू ने अपनी माँ से कहा"` teen cheezein dikhata hai: tokens, agle token ki probability, aur har layer ka attention.

10 Famous repos ke saamne hum

source code padhkar, 12 Oct 2026 ko

"LLM from scratch" ke sabse mashhoor repos ka model code padha aur apne se milaya. Interviewer aksar poochta hai "nanoGPT se alag kya kiya".

CHEEZ	NANOGPT / RASCHKA	MINIMIND	NANOCHAT	CHHOTU GPT
Kis jaisa	GPT-2 (2019)	Llama-style	2025-26 ke tricks	Llama-style
Position	seekhi hui table	RoPE	RoPE	**RoPE**
Norm	LayerNorm	RMSNorm	RMSNorm	**RMSNorm**
Feed-forward	GELU	SwiGLU	relu²	**SwiGLU**
QK norm	nahi	**haan**	**haan**	switch, default off
Weight tying	haan (nanoGPT)	haan	nahi	**haan**
Optimizer	AdamW	AdamW	Muon + AdamW	AdamW
Khaas	saaf, padhne layak	MoE option	chat stage, eval, UI	bhasha ke tags, x-ray

Ablation: unke do ideas, hamare model pe naape

Ablation study = ek cheez badlo, baaki sab same rakho, aur naapo. nanochat ke do ideas liye: **QK norm** (query aur key ko barabar lambai ka karna) aur **zero-init** (har block shuru mein "kuch nahi" karta hai).

Val loss, 500 iterations ke baad (kam = behtar)



Bars 2.6 se shuru hain taaki chhota fark dikhe; asli number bracket mein hai. Dono ideas madad karte hain aur saath mein fayda jud jaata hai (0.038 + 0.049 ≈ 0.085).

SACH BAAT

Ye **ek seed aur sirf 500 iterations** hain. Fark chhota hai, aur alag seed pe kitna hilta hai ye naapa nahi gaya. Ye bhi nahi pata ki 4000 iterations ke baad fark bachta hai. Isliye main model abhi baseline hi hai.

INTERVIEW MEIN BOLO

"I compared against the source of nanoGPT, nanochat and minimind. I took two ideas from nanochat, QK norm and zero-init projections, and ran an ablation: together they improved validation loss by 0.085 at 500 iterations. It's one seed and a short run, so it's a promising signal, not a proven result."

11 Saare 169 concepts

beginner se advanced · 153 ke saath code ki jagah, 16 sirf theory

Har term ke saamne file aur wo naam hai jo us file mein search karna hai. "theory" wale terms is project ke code mein nahi hain, par interview mein pooche jaate hain.

A. Neural network ki neenv (lesson 1: autograd.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
1	**Parameter / weight**	Wo numbers jo model seekhta hai. Model ka "gyaan" inhi mein hai.	`autograd.py self.w`
2	**Bias**	Har neuron mein juda ek extra number, jo output ko upar-neeche khiskata hai.	`autograd.py self.b`
3	**Neuron**	Inputs ko weights se guna karke jodta hai, phir activation lagata hai.	`autograd.py class Neuron`
4	**Layer**	Ek saath kaam karne wale neurons ka group.	`autograd.py self.layers`
5	**MLP**	Multi-layer perceptron: ek ke baad ek layers. Transformer ke har block mein ek hota hai.	`autograd.py class MLP`
6	**Activation function**	Non-linear function. Iske bina kitni bhi layers ek hi linear layer ke barabar hain.	`autograd.py def tanh`
7	**Forward pass**	Input se output tak numbers ko chalana.	`autograd.py def __call__`
8	**Loss**	Ek number jo batata hai prediction kitni galat hai. Training ise kam karti hai.	`autograd.py loss = sum`
9	**MSE**	Mean squared error: (prediction - target) ka square. Numbers predict karne ke liye.	`autograd.py mean squared error`
10	**Gradient**	"Is weight ko thoda badhaun to loss kitna badlega." Har weight ka apna.	`autograd.py self.grad`
11	**Computation graph**	Kaunsa number kis number se bana, uska record. Backprop isi pe chalta hai.	`autograd.py self._parents`
12	**Chain rule**	Calculus ka niyam: jude hue functions ka derivative = unke derivatives ka guna.	`autograd.py self._backward`
13	**Backpropagation**	Loss se peeche ki taraf chain rule lagakar har weight ka gradient nikalna.	`autograd.py def backward`
14	**Topological order**	Graph ko us order mein chalna ki koi node apne dependents se pehle na aaye.	`autograd.py def visit`
15	**Autograd**	Gradients apne aap nikalne wala system. PyTorch ka dil.	`autograd.py class Value`
16	**Gradient descent / SGD**	Har weight ko uske gradient ke ulte taraf thoda khiskana.	`autograd.py p.data -= lr * p.grad`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
17	**Learning rate**	Har step kitna bada ho. Zyada: loss phat jaata hai. Kam: seekhna dheema.	`autograd.py lr = 0.05`
18	**Zero grad**	Gradients jud-te jaate hain, isliye har step se pehle saaf karna padta hai.	`autograd.py p.grad = 0.0`
19	**Random initialization**	Weights random se shuru, warna saare neurons ek jaisa hi seekhenge (symmetry).	`autograd.py random.uniform`
20	**Gradient check**	Backprop ka gradient "nudge karke dekho" wale gradient se milana.	`autograd.py numeric`
21	**Divergence**	Loss ghatne ki jagah badhkar infinity ho jaana. Aksar learning rate zyada hone se.	`autograd.py divergence`
22	**Tensor**	Numbers ka multi-dimensional array. PyTorch mein sab kuch tensor hai.	`model.py torch.Tensor`

B. Text se numbers (lesson 2-3: tokenizer.py, data.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
23	**Token**	Text ka wo tukda jo model ek unit maanta hai. Akshar se bada, shabd se chhota.	`tokenizer.py def encode`
24	**Tokenizer**	Text ko token ids mein aur ids ko wapas text mein badalta hai.	`tokenizer.py class BPE`
25	**Vocabulary**	Saare possible tokens ki list. Hamari 4096 ki hai.	`tokenizer.py self.vocab`
26	**Vocab size**	Vocabulary mein kitne tokens. Bada: chhoti sequence par bada embedding table.	`tokenizer.py def vocab_size`
27	**UTF-8 / bytes**	Text ko bytes mein likhne ka tareeka. Ek Devanagari akshar = 3 bytes.	`tokenizer.py encode("utf-8")`
28	**Byte-level BPE**	256 bytes se shuru karna, taaki koi bhi script ya emoji encode ho sake.	`tokenizer.py range(256)`
29	**BPE**	Byte-pair encoding: sabse zyada saath aane wale pair ko baar-baar ek token bana dena.	`tokenizer.py def train`
30	**Merge**	Do tokens ko jodkar ek naya token banana.	`tokenizer.py def merge`
31	**Merge rank**	Merges usi order mein lagte hain jis order mein seekhe gaye.	`tokenizer.py self.rank`
32	**Pre-tokenization**	Pehle text ko shabdon mein kaatna, taaki merge shabd ke bahar na jaye.	`tokenizer.py CHUNK`
33	**Subword**	Shabd ka hissa. Naya shabd bhi purane tukdon se ban jaata hai.	`tokenizer.py def piece`
34	**OOV (out of vocabulary)**	Aisa shabd jo vocabulary mein nahi. Byte-level BPE mein ye problem hoti hi nahi.	`check.py unseen`
35	**Special token**	Haath se daale gaye control tokens, text se nahi aate.	`tokenizer.py SPECIALS`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
36	**BOS / EOS**	Document ki shuruaat / ant. EOS aate hi generation rukta hai.	`data.py bos, eos`
37	**Control token / conditional generation**	Language tag dekar model se usi bhasha mein likhwana.	`generate.py def reply`
38	**Encode / decode**	Text → ids, aur ids → text. Round trip exact hona chahiye.	`tokenizer.py def decode`
39	**Compression ratio**	Bytes per token. Achha tokenizer kam tokens mein zyada text rakhta hai.	`tokenizer.py compression`
40	**Corpus**	Training ka poora text.	`data.py def documents`
41	**Dataset**	Corpus ka source. Hamare teen: Hindi, Marathi, Hinglish.	`data.py SOURCES`
42	**TinyStories**	Chhote shabdon wali kahaniyan, jinse chhota model bhi grammar seekh leta hai.	`data.py TinyStories`
43	**Code-switching**	Ek hi vaakya mein do bhashayein (Hinglish).	`data.py Hinglish`
44	**Train / validation split**	Kuch data alag rakhna jis pe model kabhi train nahi hota.	`data.py n_val`
45	**Shuffle**	Data ka order random karna, taaki split aur batches fair hon.	`data.py shuffle`
46	**Reproducibility / seed**	Fixed seed se har baar wahi result.	`train.py manual_seed`
47	**Token ids ki binary file**	Tokens ko uint16 array mein disk pe rakhna.	`data.py np.uint16`
48	**Memory map**	Poori file RAM mein laaye bina disk se padhna.	`train.py np.memmap`
49	**WordPiece / SentencePiece / Unigram**	BPE ke rishtedaar tokenizers (BERT, T5, Gemma).	theory

C. Transformer ke andar (lesson 4: model.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
50	**Language model**	Model jo agle token ki probability deta hai.	`model.py class GPT`
51	**Transformer**	Attention + feed-forward blocks ka stack. 2017 ka "Attention Is All You Need".	`model.py self.blocks`
52	**Decoder-only**	Sirf left-to-right padhne wala transformer. GPT, Llama, Gemma sab yahi.	`model.py decoder-only`
53	**Autoregressive**	Har naya token pichhle tokens pe depend karta hai.	`generate.py autoregressive`
54	**Hyperparameter**	Jo hum chunte hain (layers, width), model nahi seekhta.	`model.py class Config`
55	**Embedding**	Token id ko seekhe hue vector mein badalna. Milte-julte tokens ke vector paas hote hain.	`model.py nn.Embedding`
56	**d_model (width)**	Har token ke vector ki lambai. Hamari 384.	`model.py d_model`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
57	**Context window**	Model ek baar mein kitne tokens dekh sakta hai. Hamara 256.	`model.py ctx: int`
58	**Depth (n_layer)**	Kitne blocks. Zyada depth = zyada steps mein sochna.	`model.py n_layer`
59	**Residual stream**	Wo vector jo blocks ke beech behta hai; har block isme kuch jodta hai.	`model.py residual stream`
60	**Residual / skip connection**	`x = x + f(x)`. Gradient seedha neeche tak pahunchta hai, isliye deep model train hota hai.	`model.py x = x + a`
61	**Vanishing / exploding gradient**	Deep network mein gradient ka bahut chhota ya bahut bada ho jaana.	`model.py class Block`
62	**Self-attention**	Har token baaki tokens se zaroori jaankari uthata hai.	`model.py class CausalSelfAttention`
63	**Query, Key, Value**	Query = sawaal, Key = label, Value = content. Query jis key se mile uski value milti hai.	`model.py self.qkv`
64	**Dot product**	Do vectors kitne milte hain uska number. Attention score yahi hai.	`model.py q @ k.transpose`
65	**Scaled dot-product**	Score ko sqrt(head_dim) se bhaag dena, warna softmax ek hi token pe atak jaata hai.	`model.py math.sqrt(self.head_dim)`
66	**Softmax**	Scores ko probabilities mein badalta hai jo milkar 1 hoti hain.	`model.py F.softmax`
67	**Attention weights**	Softmax ke baad ke numbers: kis token pe kitna dhyaan.	`model.py self.last_att`
68	**Causal mask**	Future tokens ko chhupana. Isi se ek hi pass mein har position pe training ho paati hai.	`model.py mask =`
69	**Multi-head attention**	Kai chhote attention ek saath, har ek alag rishta pakadta hai.	`model.py n_head`
70	**Head dimension**	d_model / n_head. Har head ke vector ki lambai.	`model.py self.head_dim`
71	**Output projection**	Saare heads ko wapas ek vector mein milana.	`model.py self.proj`
72	**Positional encoding**	Attention ko order nahi pata, isliye position alag se batani padti hai.	`model.py def rope_table`
73	**RoPE**	Query aur key ko position ke hisaab se ghumana; dot product sirf doori pe depend karta hai.	`model.py def apply_rope`
74	**Absolute vs relative position**	"Main 5th token hun" banam "wo mujhse 3 peeche hai". RoPE relative deta hai.	`model.py relative position`
75	**Feed-forward network**	Har token pe alag se chalne wala MLP. Zyadatar parameters yahin.	`model.py class FeedForward`
76	**SiLU / Swish**	`x * sigmoid(x)`. ReLU ka chikna roop.	`model.py F.silu`
77	**SwiGLU**	Gate wala feed-forward: ek branch doosri ko control karti hai. Llama, Gemma mein.	`model.py self.gate`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
78	**ReLU / GELU**	Purane activations. Original transformer ReLU, GPT-2 GELU.	theory
79	**Normalization**	Vectors ka size sthir rakhna, taaki training na bigde.	`model.py class RMSNorm`
80	**LayerNorm vs RMSNorm**	RMSNorm = LayerNorm bina mean aur bias ke. Sasta, utna hi achha.	`model.py self.gain`
81	**Pre-norm vs post-norm**	Norm sub-layer se pehle (aaj) ya baad mein (2017). Pre-norm zyada stable.	`model.py self.norm1`
82	**Transformer block**	Norm → attention → add, norm → feed-forward → add.	`model.py class Block`
83	**Logits**	Softmax se pehle ke raw scores, har vocab entry ka ek.	`model.py logits = self.lm_head`
84	**LM head / unembedding**	Aakhri vector ko vocab-size scores mein badalne wali layer.	`model.py self.lm_head`
85	**Weight tying**	Input embedding aur output head ek hi matrix. Parameters bachte hain.	`model.py self.lm_head.weight = self.tok_emb.weight`
86	**Weight initialization**	Shuruaati random values ka size. Galat ho to training shuru hi nahi hoti.	`model.py nn.init.normal_`
87	**Parameter count**	Model ka size. Hamara 12.2 million.	`model.py def n_params`
88	**Buffer**	Model ke saath chalne wala tensor jo train nahi hota (RoPE table).	`model.py register_buffer`
89	**Batch / time / channel (B, T, C)**	Tensor ke teen dimension: kitni sequences, kitne tokens, vector ki lambai.	`model.py B, T, C`
90	**Encoder / encoder-decoder**	BERT sirf encoder (dono taraf dekhta), T5 dono. STT mein encoder milega.	theory
91	**Cross-attention**	Ek sequence doosri sequence pe attention. Whisper aur TTS mein.	theory
92	**Flash attention / GQA / MQA**	Attention ko tez ya sasta karne ke tareeke.	theory

D. Training (lesson 5: train.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
93	**Self-supervised learning**	Label khud text se aata hai: agla token. Kisi insaan ne label nahi kiya.	`train.py self-supervised`
94	**Pre-training**	Bahut saare text pe bhasha seekhna.	`train.py def main`
95	**Next-token prediction**	Training ka poora lakshya.	`train.py def get_batch`
96	**Teacher forcing**	Training mein model ko sahi pichhle tokens dena, uske apne guess nahi.	`model.py Teacher forcing`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
97	**Cross-entropy loss**	Sahi token ko di gayi probability ka negative log.	`model.py def cross_entropy`
98	**Log-softmax / logsumexp**	Softmax ka log, numerically stable tareeke se.	`model.py logsumexp`
99	**Perplexity**	exp(loss). "Model itne tokens ke beech confuse hai."	`train.py math.exp(va)`
100	**Starting loss = ln(vocab)**	Random model ka loss. Isse pata chalta hai setup sahi hai.	`train.py math.log(tok.vocab_size)`
101	**Batch / batch size**	Ek step mein kitni sequences ek saath.	`train.py --batch`
102	**Iteration / step**	Ek batch pe ek update.	`train.py for it in range`
103	**Epoch**	Poore data pe ek chakkar.	`train.py epochs`
104	**Optimizer**	Gradient se weights update karne ka niyam.	`train.py opt.step()`
105	**Adam / AdamW**	Momentum + har weight ka apna step size. W = sahi tareeke ka weight decay.	`train.py AdamW`
106	**Momentum / betas**	Pichhle gradients ka chalta hua average.	`train.py betas`
107	**Weight decay**	Weights ko zero ki taraf kheenchna. Ek regularizer.	`train.py weight_decay`
108	**Regularization**	Jo cheez overfitting rokti hai (dropout, weight decay).	`model.py dropout`
109	**Dropout**	Training mein kuch numbers random zero karna, taaki model kisi ek pe na tike.	`model.py nn.Dropout`
110	**LR schedule**	Training ke dauran learning rate badalna.	`train.py def lr_at`
111	**Warmup**	Shuru mein learning rate dheere badhana.	`train.py warmup`
112	**Cosine decay**	Learning rate ko cosine curve pe ghatana.	`train.py math.cos`
113	**Gradient clipping**	Bahut bade gradient ko chhota kar dena.	`train.py clip_grad_norm_`
114	**Gradient norm**	Saare gradients ki kul lambai. Training ki sehat ka meter.	`train.py grad_norm`
115	**Train loss vs val loss**	Train: seekhe hue data pe. Val: andekhe data pe.	`train.py def estimate_loss`
116	**Overfitting**	Train loss girta rahe, val loss badhne lage: model ratta maar raha hai.	`train.py overfitting`
117	**Underfitting**	Dono loss oonche: model chhota hai ya kam train hua.	theory
118	**Generalization**	Andekhe text pe achha karna. Asli lakshya.	`data.py generalization`
119	**Checkpoint**	Training ke beech save kiya gaya model.	`model.py def save`
120	**Early stopping**	Sabse achhe val loss wala checkpoint rakhna.	`train.py if va < best`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
121	**train() / eval() mode**	Dropout on / off.	`train.py model.eval()`
122	**Device (CPU / GPU / MPS)**	Hisaab kahan hota hai. MPS = Mac ka GPU.	`model.py def pick_device`
123	**Throughput (tokens/sec)**	Training ki speed.	`train.py tok/s`
124	**Gradient accumulation**	Kai chhote batches ke gradient jodkar ek bada batch banana.	theory
125	**Mixed precision (AMP)**	fp16 mein train karna, tez aur kam memory.	theory
126	**Scaling laws / compute budget**	Data, parameters aur compute ka rishta.	theory

E. Generation (lesson 6: generate.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
127	**Inference**	Train hue model ko use karna.	`generate.py torch.no_grad`
128	**Decoding / sampling**	Probabilities se ek token chunna.	`generate.py def sample`
129	**Greedy decoding**	Hamesha sabse likely token.	`generate.py argmax`
130	**Temperature**	Logits ko T se bhaag. Kam = safe, zyada = creative.	`generate.py temperature`
131	**Top-k**	Sirf k sabse likely tokens rakhna.	`generate.py top_k`
132	**Top-p / nucleus**	Utne tokens rakhna jinki probability milkar p ho.	`generate.py top_p`
133	**Multinomial sampling**	Probabilities ke hisaab se pasa phenkna.	`generate.py torch.multinomial`
134	**Repetition penalty**	Pehle aa chuke tokens ko dabana.	`generate.py repetition_penalty`
135	**Stop token**	Jis token pe generation ruk jaye.	`generate.py stop`
136	**KV cache**	Purane tokens ke keys / values yaad rakhna, dobara na nikalna.	`model.py past`
137	**Prompt**	Wo text jisse generation shuru hota hai.	`generate.py prompt`
138	**Prompt format / chat template**	Prompt ka wahi dhaancha jo training data ka tha.	`generate.py def reply`
139	**Hallucination**	Model ka poore vishwas se galat baat kehna. Wo sach nahi, agla token predict karta hai.	theory
140	**Beam search**	Kai raaste saath rakhkar sabse achha chunna. STT mein milega.	theory

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
141	**Time to first token (TTFT)**	Pehla token aane mein laga samay. Voice agent mein sabse zaroori.	theory
142	**Streaming**	Tokens bante hi bhejna.	theory

F. Fine-tuning aur chhota karna (lesson 7-9: lora.py, shrink.py, xray.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
143	**Fine-tuning**	Train hue model ko naye kaam pe thoda aur train karna.	`lora.py def main`
144	**Transfer learning**	Ek kaam ka gyaan doosre kaam mein lagana.	`lora.py Pre-training`
145	**Full fine-tuning**	Saare weights update karna. Mehenga.	`lora.py Full fine-tuning`
146	**PEFT**	Parameter-efficient fine-tuning: thode se weights train karna.	`lora.py def add_lora`
147	**LoRA**	Frozen W ke saath chhota B·A jodna.	`lora.py class LoRALinear`
148	**Rank (r)**	A aur B kitne patle hain. Chhota r = kam parameters.	`lora.py r: int`
149	**Alpha / scale**	LoRA ke update ki taakat: alpha / r.	`lora.py self.scale`
150	**Frozen weights**	Jo train nahi hote.	`lora.py requires_grad_(False)`
151	**Adapter**	Sirf naye weights wali chhoti file.	`lora.py persona_adapter.pt`
152	**Merge**	B·A ko W mein jod dena; speed wapas wahi.	`lora.py def merge`
153	**Padding**	Chhoti sequences ko lamba karna taaki batch chaukor bane.	`lora.py Padding`
154	**Ignore index**	Padding wali positions ko loss se bahar rakhna.	`lora.py -100`
155	**Catastrophic forgetting**	Naya seekhte hue purana bhool jaana.	`lora.py catastrophic forgetting`
156	**Quantization**	Weights ko kam bits mein rakhna.	`shrink.py def quant_roundtrip`
157	**fp32 / fp16 / int8**	4, 2, 1 byte per weight.	`shrink.py w.half()`
158	**Scale (quantization)**	Float aur integer ke beech ka guna-bhaag.	`shrink.py scale`
159	**Per-tensor vs per-channel**	Poori matrix ka ek scale banam har row ka apna.	`shrink.py per_channel`
160	**PTQ vs QAT**	Training ke baad quantize banam training ke dauran. Gemma QAT hai.	`shrink.py post-training quantization`
161	**GGUF**	llama.cpp ka file format, quantized models ke liye.	theory

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
162	**Distillation**	Bade model se chhote model ko sikhana.	theory
163	**Entropy**	Model kitna unsure hai, bits mein.	`xray.py entropy`
164	**Interpretability**	Model ke andar dekhkar samajhna ki wo kya kar raha hai.	`xray.py last_att`
165	**RAG / instruction tuning / RLHF**	Bade LLMs ko kaam ka banane ke tareeke.	theory
166	**QK norm**	Query aur key ko barabar lambai ka karna; score sirf disha (cosine similarity) naapta hai.	`model.py qk_norm`
167	**Zero-init projection**	Har block ka output shuru mein zero, to naya model pehle "kuch nahi" karta hai.	`model.py zero_init`
168	**Ablation study**	Ek cheez badlo, baaki same rakho, aur naapo. Dekho COMPARE.md.	`train.py --qk-norm`
169	**Replay / rehearsal**	Fine-tune ke saath purana data bhi dikhana, taaki model bhoole nahi.	`lora.py Replay`

12 Interview: sawaal aur jawab

"BOLO" wali line English mein seedhe boli ja sakti hai

30 SECOND KA PITCH

"I built a decoder-only transformer from scratch in PyTorch to really understand how LLMs work. I wrote my own byte-level BPE tokenizer, multi-head causal self-attention with RoPE, RMSNorm and SwiGLU, and a training loop with AdamW, warmup, cosine decay and gradient clipping. I trained a 12 million parameter model on Hindi, Marathi and Hinglish, taking loss from 8.37 to 1.88. Then I added a KV cache, LoRA fine-tuning and int8 quantization, and wrote tests for causality and cache correctness."

A. Buniyaad

LLM asal mein kya karta hai?

BOLO It predicts a probability distribution over the next token given all previous tokens. Text is generated by sampling one token and feeding it back in.

Bas agla token. Baaki sab (jawab dena, kahani likhna) isi ko baar-baar chalane se aata hai.

Token kya hai? Shabd kyun nahi use karte?

BOLO A token is a subword unit. Word-level needs a huge vocabulary and fails on unseen words; character-level makes sequences too long. Subwords are the trade-off.

Aam shabd ek token, mushkil shabd tukdon mein. Naya shabd bhi purane tukdon se ban jaata hai.

BPE kaise kaam karta hai?

BOLO Start with bytes as the vocabulary, repeatedly merge the most frequent adjacent pair into a new token until you reach the target vocabulary size.

Hamne 3,835 merges kiye. Encode karte waqt merges usi order mein lagte hain jis order mein seekhe gaye.

Embedding kya hai?

BOLO A learned lookup table that maps each token id to a dense vector. Tokens used in similar contexts end up with similar vectors.

4096 × 384 ki ek table. Row number = token id.

B. Attention

Self-attention samjhao.

BOLO Each token builds a query, a key and a value. The score between two tokens is the dot product of one's query and the other's key. Softmax turns scores into weights, and the output is the weighted sum of values.

Query = sawaal, key = label, value = maal. Jiski key meri query se mile, uska maal mujhe zyada.

√d se bhaag kyun dete hain?

BOLO Dot products grow with dimension. Large scores push softmax into saturation where gradients vanish. Scaling keeps the variance around one.

Bina iske softmax ek hi token ko sab kuch de deta hai aur seekhna ruk jaata hai.

Multi-head kyun?

BOLO One head can only express one attention pattern. Multiple heads let the model attend to different relations in parallel at the same total cost.

Hamare 6 heads, har ek 64 dimension ka. 6 × 64 = 384.

Causal mask kyun zaroori hai?

BOLO It stops a position from seeing future tokens. That lets us train on every position of a sequence in one forward pass without leaking the answer.

Bina mask ke model agla token dekh ke copy kar leta, kuch nahi seekhta. Hamara test ise check karta hai.

Position ki jaankari kaise dete ho? RoPE kya hai?

BOLO Attention is permutation-invariant, so position must be injected. RoPE rotates query and key vectors by an angle proportional to position, so their dot product depends only on relative distance.

Purana tareeka: har position ka seekha hua vector jodna. RoPE mein alag table nahi chahiye.

Attention ki complexity kya hai?

BOLO Quadratic in sequence length, $O(T^2)$, because every token attends to every earlier token. That is why context windows are expensive.

256 tokens = 65,536 pair. 2,560 tokens = 100 guna zyada.

C. Architecture

Residual connection kyun?

BOLO It gives gradients a direct path through the network, so deep stacks train. Each block only has to learn a correction to its input.

`x = x + f(x)`. Bina iske 6 layers ke baad gradient lagbhag gayab ho jaata.

Normalization kyun? Pre-norm aur post-norm mein fark?

BOLO Normalization keeps activation scale stable. Pre-norm applies it before each sub-layer so the residual path stays an identity, which trains more stably than the original post-norm.

Hamne RMSNorm liya: LayerNorm bina mean aur bias ke.

Feed-forward layer ka kya kaam hai?

BOLO Attention mixes information across tokens; the feed-forward layer transforms each token independently. It holds most of the parameters.

Hamare model mein 7.08 M of 12.2 M parameters feed-forward mein hain.

Weight tying kya hai?

BOLO The input embedding and the output projection share one matrix. It saves parameters and usually helps small models.

1.57 M parameters bache. nanochat ise nahi karta, kyunki bade model mein alag rakhna behtar nikla.

GPT aur BERT mein fark?

BOLO GPT is decoder-only with causal attention, trained to predict the next token, so it generates. BERT is encoder-only with bidirectional attention, trained on masked tokens, so it is for understanding.

Hamara model GPT jaisa hai. STT ke module mein encoder milega.

D. Training

Loss function kya hai?

BOLO Cross-entropy: the negative log probability the model assigned to the correct next token, averaged over all positions.

Sahi token ko 90% diya to loss 0.1; 1% diya to 4.6.

Training shuru karte hi kya check karte ho?

BOLO That the initial loss is about ln of the vocabulary size. For 4096 that is 8.32, and I got 8.37. If it is far off, there is a bug in the model or the data.

Sabse sasta aur sabse kaam ka check.

Perplexity kya hai?

BOLO The exponential of the loss. It reads as the effective number of choices the model is uncertain between. Mine is 6.5.

Random model ka 4,096 hota.

AdamW kyun, SGD kyun nahi?

BOLO Adam keeps a per-parameter step size from running gradient statistics, which transformers need because gradient scales differ a lot between layers. The W means weight decay is applied directly to the weights instead of through the gradient.

Weight decay sirf matrices pe lagaya, norm ke gains pe nahi.

Warmup aur cosine decay kyun?

BOLO Early gradients from a random network are noisy, so we ramp the learning rate up. Then we decay it so the model settles into a minimum instead of bouncing around.

200 step warmup, phir 6e-4 se 6e-5 tak.

Overfitting kaise pakdi?

BOLO I hold out 2% of documents and compare training and validation loss. If training keeps dropping while validation rises, it is memorizing. In my run the gap stayed at 0.19.

Aur sabse achhe val loss wala checkpoint hi save hota hai.

E. Inference

Temperature, top-k, top-p samjhao.

BOLO Temperature rescales logits to make the distribution sharper or flatter. Top-k keeps the k most likely tokens. Top-p keeps the smallest set whose cumulative probability reaches p, so it adapts to the model's confidence.

Chapter 6 ka chart dekho: wahi prompt, teen temperature.

KV cache kya hai?

BOLO During generation the keys and values of earlier tokens never change, so we cache them and only process the new token. It turns each step from re-reading the whole sequence into one token's work.

Is machine pe 3.2x tez, aur test ye pakka karta hai ki output wahi hai.

Model galat baat kyun bolta hai (hallucination)?

BOLO It is trained to produce likely text, not true text. It has no mechanism to check facts, so a fluent wrong continuation is as valid to it as a correct one.

Hamare model mein ye saaf dikhta hai: grammar sahi, kahani ka logic toota hua.

F. Fine-tuning aur deployment

LoRA kya hai? Full fine-tuning se behtar kyun?

BOLO LoRA freezes the base weights and trains a low-rank update. It needs far less memory and storage, and one base model can serve many small adapters.

Hamara adapter 0.45 MB, poora model 48.8 MB.

LoRA mein B ko zero se kyun shuru karte hain?

BOLO So that B times A is zero at the start and the model behaves exactly like the pretrained one. Training then moves away from that smoothly.

Hamara test: naya adapter lagane pe output bilkul same.

Catastrophic forgetting kya hai? Tumne dekha?

BOLO When fine-tuning on new data damages what the model already knew. I saw general validation loss rise from 2.70 to 3.19, and reduced it by mixing a batch of original data into each step.

Is technique ko replay ya rehearsal kehte hain.

Quantization kya hai? Kitna nuksaan hua?

BOLO Storing weights in fewer bits. int8 cost nothing on my model, 4-bit per-channel cost 0.03 in loss, and 2-bit collapsed.

Per-channel = har row ka apna scale. 4-bit pe ye per-tensor se 4 guna behtar tha.

G. Project ke baare mein

Kaise pata ki tumhara code sahi hai?

BOLO I wrote tests for the properties that must hold: changing a future token does not change earlier logits, the KV cache gives identical logits to a full pass, my cross-entropy matches PyTorch's, and a fresh LoRA adapter is a no-op.

Loss girna kaafi nahi; galat model bhi kuch na kuch seekh leta hai.

Sabse mushkil ya dilchasp bug kaunsa tha?

BOLO Quantization showed zero change in loss, which looked like a bug. I measured the actual weight error and found int8 was only about 1% off, so the result was real. I then pushed to 4, 3 and 2 bits to find where it breaks.

Seekh: "kuch nahi badla" ko pehle shak se dekho, phir naap ke maano.

nanoGPT se alag kya kiya?

BOLO nanoGPT reproduces GPT-2. Mine uses the modern stack: RoPE, RMSNorm, SwiGLU. I also wrote the tokenizer, added language control tokens for Hindi, Marathi and Hinglish, a KV cache, LoRA, quantization measurements and an attention inspector.

Aur ek ablation study jo nanochat ke do ideas ko naapti hai.

Kya ye production mein use ho sakta hai?

BOLO No. With 12 million parameters and 11 million tokens it writes simple stories and short chat, not facts or reasoning. Its purpose was understanding. For real use I would fine-tune an existing model.

Ye sachcha jawab hai, aur ye tumhe samajhdaar dikhata hai.

Aage kya karoge?

BOLO Run the QK-norm and zero-init variant for the full length with multiple seeds, add more data since validation loss was still falling, add a chat fine-tuning stage with role tokens, and build a fixed evaluation set beyond loss.

Phir STT, TTS aur orchestration, isi tarah scratch se.

13 Ek page ki cheat sheet

interview se 5 minute pehle

12.2 M parameters	**6 / 6 / 384** layers / heads / width	**256** context window	**4,096** vocab (byte-level BPE)
10.9 M train tokens, 4.5 epochs	**1.88** val loss (shuru 8.37, ln 4096 = 8.32)	**6.5** val perplexity	**~50 min** M1 Pro ke GPU pe
3.2x KV cache se speed	**0.91%** LoRA mein train hue weights	**12 MB** int8 mein (fp32: 49 MB), loss mein fark 0	**−0.085** QK norm + zero-init (500 iter, 1 seed)

Paanch line jo yaad rakhni hain

- **LLM** = agle token ki probability. Text = ise baar-baar chalana.
- **Attention** = softmax(Q·K / √d) × V, future pe mask ke saath.
- **Block** = x + attention(norm(x)), phir x + feed_forward(norm(x)).
- **Training** = cross-entropy + AdamW + warmup/cosine + clipping, val loss pe nazar.
- **Deploy** = KV cache (tez), LoRA (sasta fine-tune), quantization (chhota).

Khud chalao

KYA DEKHNA HAI	COMMAND (FOLDER: `SCRATCH/LLM`)
Saare tests	`python check.py`
Hindi kahani	`python generate.py --lang hi --prompt "एक दिन"`
Hinglish chat	`python generate.py --lang hing --chat --prompt "kya haal hai?"`
Model ke andar	`python xray.py --lang hi --prompt "एक दिन राजू ने"`
Ye guide dobara banao	`python docs/build.py`

SABSE ACHHI TAYYARI

Padhna kaafi nahi. `autograd.py` mein learning rate badha kar dekho. `model.py` mein mask hata kar `check.py` chalao aur dekho kaunsa test fail hota hai. Jo cheez khud todi hai, wo yaad rehti hai.

IS MODEL KI SEEMA

Chhota (12 M), kam data (11 M tokens, machine ki likhi kahaniyan), 256 tokens tak, facts nahi jaanta, ablation ek seed ka. Ye sab bolna theek hai: maksad samajhna tha.